

MATERIAL DO ALUNO

Guia de Live Coding — ALUNO

Siga o passo a passo e crie seu primeiro projeto React (o cardápio TechFood). Cada passo tem a imagem de como deve ficar na sua tela. Marque as caixinhas ao concluir. Travou? Chame o professor — errar faz parte!

Como usar

Os blocos escuros são o que você digita. As imagens mostram como deve ficar. Onde aparecer **VOCÊ FAZ**, é a sua vez. Vá no ritmo do professor.

Antes de começar

Confira se o Node está instalado. No terminal digite `node -v` — tem que aparecer um número de versão. Se não aparecer, avise o professor.

1 Abrir o terminal na pasta do projeto

VOCÊ FAZ — digite no terminal, uma linha por vez:

terminal

```
cd Desktop
mkdir react-techfood
cd react-techfood
```

- ☐ Criei/entrei na pasta react-techfood.

2 Criar o projeto React com o Vite

VOCÊ FAZ — digite (não esqueça o ponto no final):

terminal

```
npm create vite@latest .
```

O Vite faz 4 perguntas. Responda com as setas e Enter:

- Select a framework React

```
Problemas Saída Console de Depuração Terminal Postman Console
PS D:\DOCUMENTOS\SENAI\FRONT-END\PRE_AULA\REACT\AULA_03\react-techfood> npm create vite@latest .
> npx
> create-vite .

|
◆ Select a framework:
  ○ Vanilla
  ○ Vue
  ● React
  ○ Preact
  ○ Lit
  ○ Svelte
  ○ Solid
  ○ Ember
  ○ Qwik
  ○ Angular
  ○ Marko
  ○ Others
  ↑/↓ to navigate • Enter: confirm
```

1) Escolha React.

- Select a variant JavaScript

```
Problemas Saída Console de Depuração Terminal Postman Console
PS D:\DOCUMENTOS\SENAI\FRONT-END\PRE_AULA\REACT\AULA_03\react-techfood> npm create vite@latest .
> npx
> create-vite .

|
◇ Select a framework:
  React
|
◆ Select a variant:
  ○ TypeScript
  ○ TypeScript + React Compiler
  ● JavaScript
  ○ JavaScript + React Compiler
  ○ RSC
  ○ React Router v7 ↗ https://reactrouter.com
  ○ TanStack Router ↗ https://tanstack.com/router
  ○ RedwoodSDK ↗ https://rwsdk.com
  ○ Vike ↗ https://vike.dev
  ↑/↓ to navigate • Enter: confirm
```

2) Escolha JavaScript.

- Which linter to use? ESLint

```

Problemas Saída Console de Depuração Terminal Postman Console
PS D:\DOCUMENTOS\SENAI\FRONT-END\PRE_AULA\REACT\AULA_03\react-techfood> npm create vite@latest .

> npx
> create-vite .

|
◇ Select a framework:
  React
◇ Select a variant:
  JavaScript
◆ Which linter to use?
  ● Oxlint
  ○ ESLint
  ↑/↓ to navigate • Enter: confirm

```

3) Escolha ESLint (aponta erros e boas práticas enquanto você digita).

- Install with npm and start now? siga o que o professor pedir (No na aula, pra você digitar os comandos)

```

Problemas Saída Console de Depuração Terminal Postman Console
PS D:\DOCUMENTOS\SENAI\FRONT-END\PRE_AULA\REACT\AULA_03\react-techfood> npm create vite@latest .

> npx
> create-vite .

|
◇ Select a framework:
  React
◇ Select a variant:
  JavaScript
◇ Which linter to use?
  ESLint
◆ Install with npm and start now?
  ● Yes / ○ No

```

4) Instalar agora? Na aula, escolha No para praticar os comandos.

O que é o Vite?

O Vite (“rápido” em francês) é a ferramenta que cria e roda o projeto React, com a tela atualizando na hora quando você salva. É o padrão usado hoje no lugar do antigo Create React App.

- Respondi as 4 perguntas do Vite.

3 Instalar e rodar

VOCÊ FAZ — se escolheu No, digite os dois comandos (se escolheu Yes, já rodou sozinho):

terminal

```

npm install
npm run dev

```

```

Problemas  Saída  Console de Depuração  Terminal  Postman Console
PS D:\DOCUMENTOS\SENAI\FRONT-END\PRE_AULA\REACT\AULA_03\react-techfood> npm create vite@latest .

> npx
> create-vite .

|
◇ Select a framework:
|  React
◇ Select a variant:
|  JavaScript
◇ Which linter to use?
|  ESLint
◇ Install with npm and start now?
|  Yes
◇ Scaffolding project in D:\DOCUMENTOS\SENAI\FRONT-END\PRE_AULA\REACT\AULA_03\react-techfood...
|
◇ Installing dependencies with npm...
:::

```

Instalando as dependências (cria a pasta node_modules).

Quando terminar, aparece o endereço <http://localhost:5173/>. Segure Ctrl e clique nele.

```

Problemas  Saída  Console de Depuração  Terminal  Postman Console
|
|  ESLint
◇ Install with npm and start now?
|  Yes
◇ Scaffolding project in D:\DOCUMENTOS\SENAI\FRONT-END\PRE_AULA\REACT\AULA_03\react-techfood...
|
◇ Installing dependencies with npm...

added 143 packages, and audited 144 packages in 14s

31 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
◇ Starting dev server...

> react-techfood@0.0.0 dev
> vite

VITE v8.3.0 ready in 618 ms
→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help

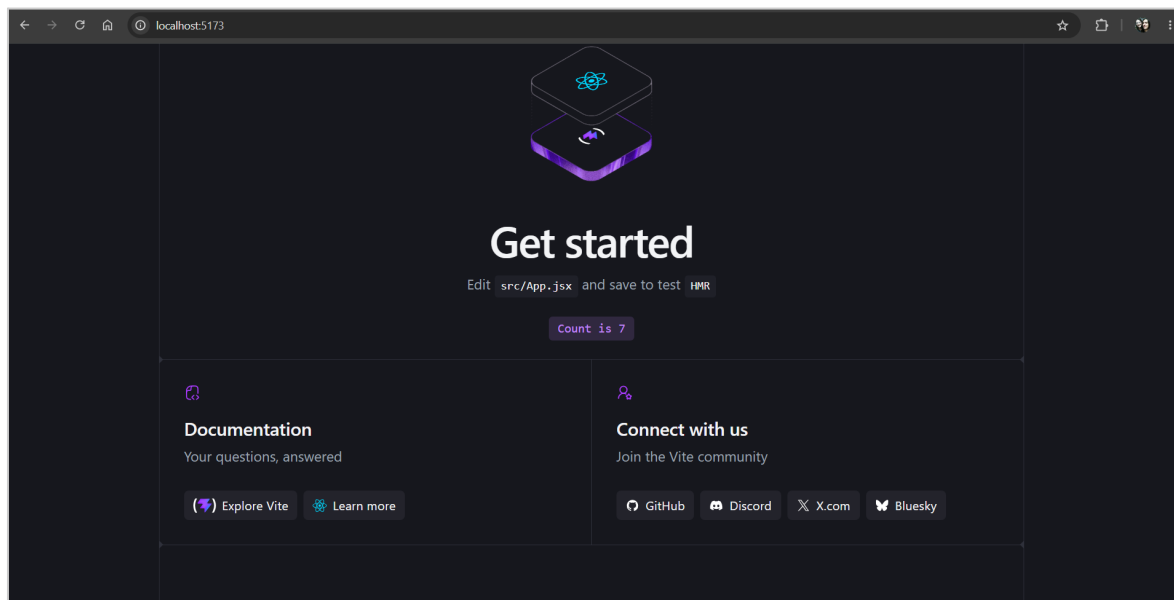
```

Servidor pronto: VITE ready + Local: <http://localhost:5173/>

- ❑ Meu servidor subiu e mostrou o localhost:5173.

4 Ver a página inicial do Vite

No navegador vai abrir a página padrão do Vite + React. Clique no botão count e veja o número subir — isso já é React reagindo!



Página inicial do Vite + React (Get started / count).

- Minha página React abriu no navegador.

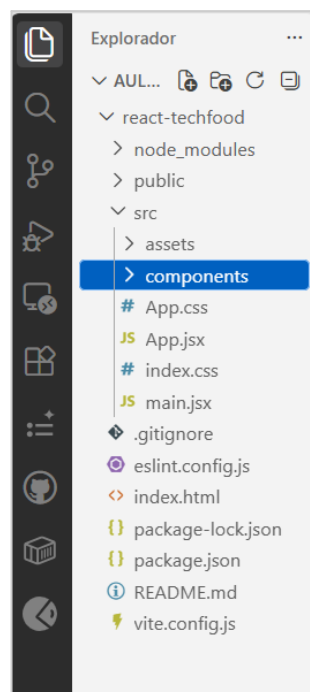
Não feche o terminal

Ele fica rodando o servidor a aula inteira. É assim mesmo.

5

Abrir no VS Code e conhecer a estrutura

Abra a pasta no VS Code. O arquivo principal que vamos editar é o `src/App.jsx`. Vamos criar nossos componentes dentro de `src/components/`.



Estrutura do projeto: `src/`, `components/`, `App.jsx`, `eslint.config.js...`

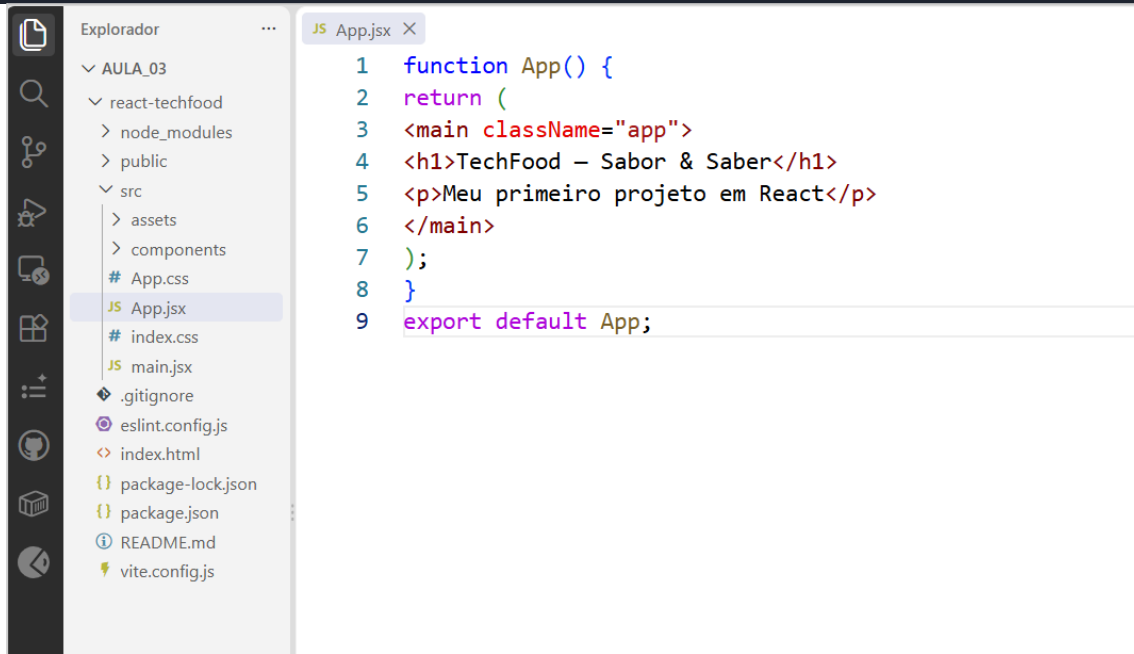
- Abri o projeto no VS Code e achei o `src/App.jsx`.

6 Escrever seu primeiro “Olá” em React

VOCÊ FAZ — apague tudo do `src/App.jsx` e escreva:

`src/App.jsx`

```
function App() {  
  return (  
    <main className="app">  
      <h1>TechFood – Sabor & Saber</h1>  
      <p>Meu primeiro projeto em React</p>  
    </main>  
  );  
}  
  
export default App;
```



Seu App.jsx editado.

Salve (Ctrl+S) e veja a tela mudar sozinha (isso é o HMR):



Resultado no navegador.

O que aprendeu aqui

Isso que parece HTML dentro do JS é o JSX. Use `className` (não `class`) e use tags com significado: `<main>`, `<header>`, `<section>`. Tudo do `return` fica dentro de um elemento pai.

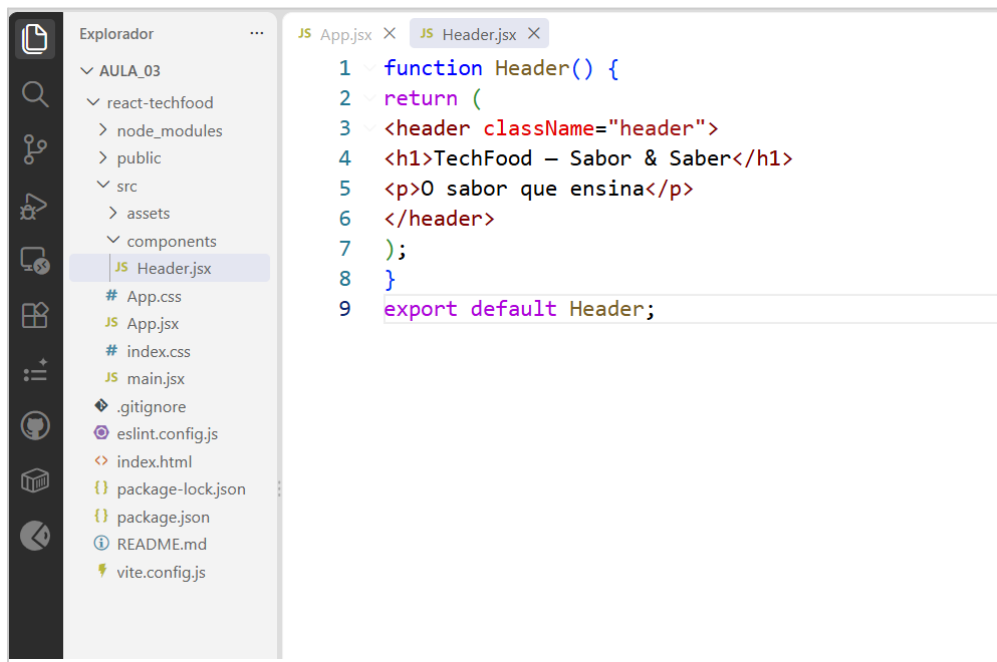
- ☐ Minha tela mostra o título da TechFood.

7 Criar seu primeiro componente — Header.jsx

VOCÊ FAZ — crie o arquivo `src/components/Header.jsx`:

`src/components/Header.jsx`

```
function Header() {  
  return (  
    <header className="header">  
      <h1>TechFood – Sabor & Saber</h1>  
      <p>O sabor que ensina</p>  
    </header>  
  );  
}  
  
export default Header;
```

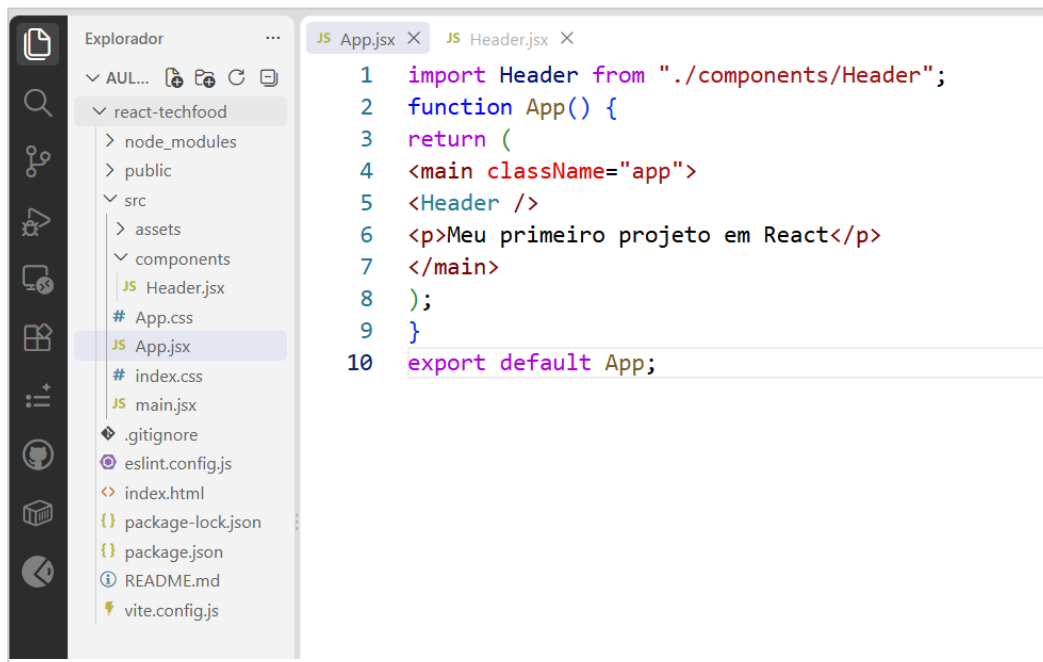


Seu Header.jsx.

Agora use ele no App.jsx: importe no topo e coloque como uma tag <Header />:

src/App.jsx

```
import Header from "../components/Header";  
  
function App() {  
  return (  
    <main className="app">  
      <Header />  
      <p>Meu primeiro projeto em React</p>  
    </main>  
  );  
}  
  
export default App;
```



App.jsx importando e usando o Header.



O Header aparece na tela.

O que é um componente

Uma função com nome em Maiúscula (PascalCase) que retorna JSX e termina com export default. É uma peça reutilizável — você usa como tag: `<Header />`.

- Criei o Header e usei `<Header />` no App.

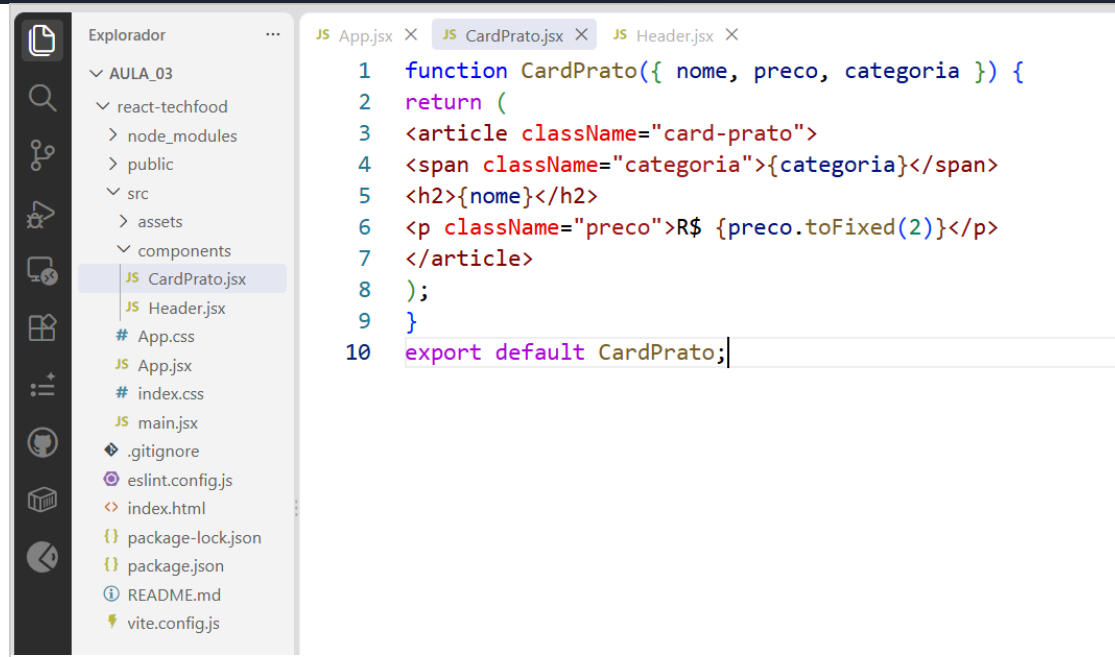
8 Componente com props — CardPrato.jsx

VOCÊ FAZ — crie `src/components/CardPrato.jsx`. Ele recebe dados de fora (props):

```
src/components/CardPrato.jsx
```

```
function CardPrato({ nome, preco, categoria }) {
  return (
    <article className="card-prato">
      <span className="categoria">{categoria}</span>
      <h2>{nome}</h2>
      <p className="preco">R$ {preco.toFixed(2)}</p>
    </article>
  );
}

export default CardPrato;
```

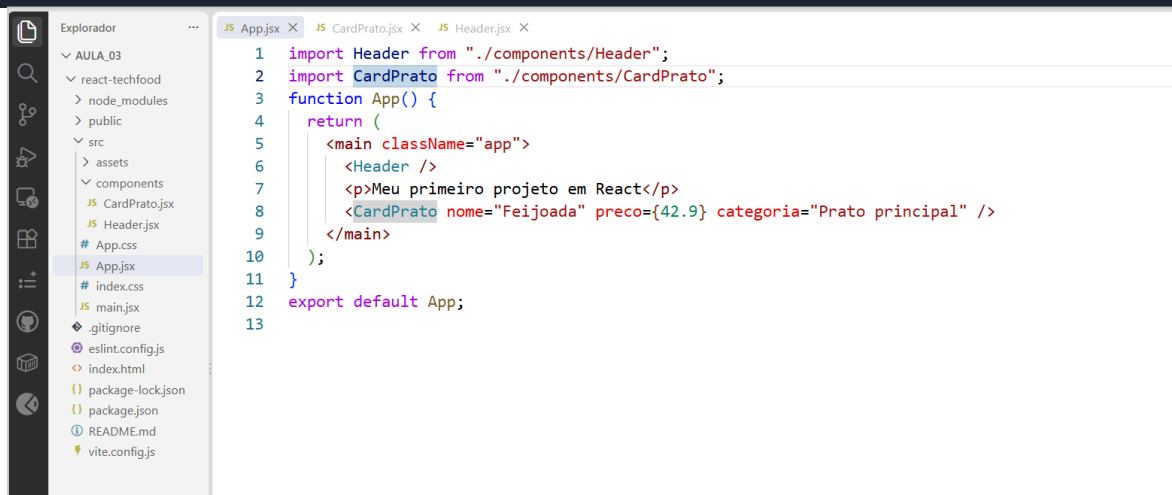


Seu CardPrato.jsx (recebe nome, preco, categoria).

Use um no App.jsx, passando os dados (as props parecem atributos de tag):

src/App.jsx (dentro do return)

```
<CardPrato nome="Feijoada" preco={42.90} categoria="Prato principal" />
```



App.jsx usando um CardPrato.



Um card aparece com os dados que você passou.

O que são props

São os dados que o pai passa pro filho. Texto usa aspas (nome="Feijoada"); número/JS usa chaves (preco={42.90}). O componente é a fôrma; as props são o recheio.

- Criei o CardPrato e mostrei um prato na tela.

9 Lista de pratos — array + .map()

VOCÊ FAZ — no App.jsx, crie o cardápio e gere um CardPrato pra cada item:

```
src/App.jsx
```

```

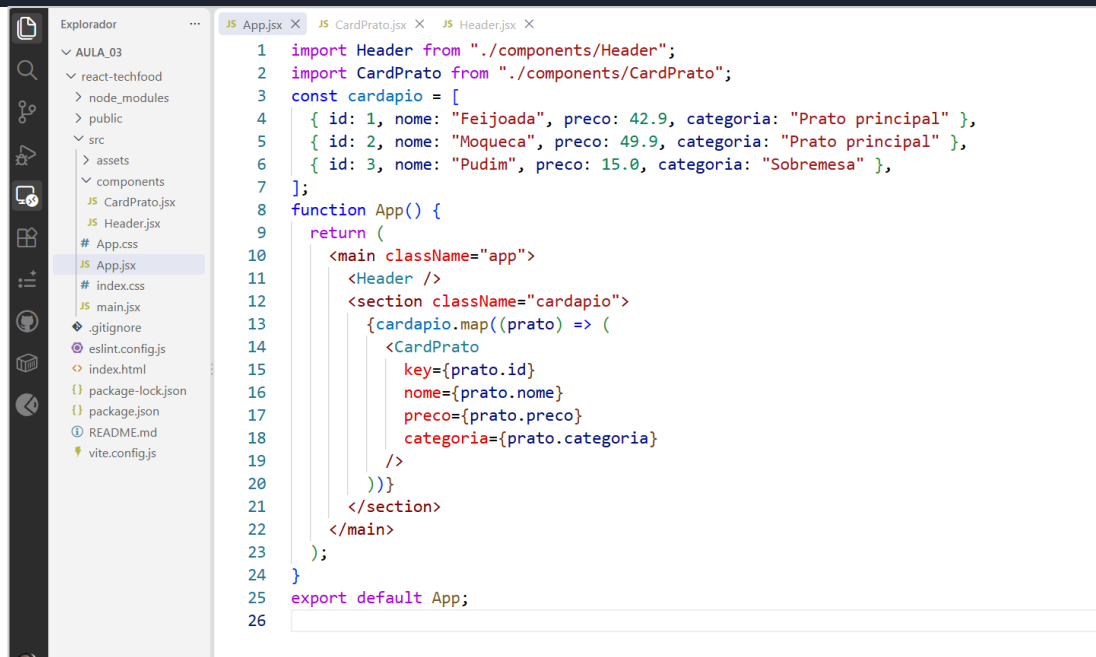
import Header from "../components/Header";
import CardPrato from "../components/CardPrato";

const cardapio = [
  { id: 1, nome: "Feijoada", preco: 42.90, categoria: "Prato principal" },
  { id: 2, nome: "Moqueca", preco: 49.90, categoria: "Prato principal" },
  { id: 3, nome: "Pudim", preco: 15.00, categoria: "Sobremesa" },
];

function App() {
  return (
    <main className="app">
      <Header />
      <section className="cardapio">
        {cardapio.map((prato) => (
          <CardPrato
            key={prato.id}
            nome={prato.nome}
            preco={prato.preco}
            categoria={prato.categoria}
          />
        ))}
      </section>
    </main>
  );
}

export default App;

```



App.jsx final: array cardapio + .map() com key.



Os 3 pratos aparecem, cada um num card.

.map() e key

O `.map()` cria um componente por item do array. O `key={prato.id}` dá identidade única a cada item — use sempre um id, nunca o índice.

- Meu cardápio completo aparece (Feijoada, Moqueca, Pudim).

10

BÔNUS — deixar o preço em real (R\$ 42,90)

VOCÊ FAZ — no `CardPrato.jsx`, calcule o preço formatado antes do `return` e use ele:

`src/components/CardPrato.jsx`

```
function CardPrato({ nome, preco, categoria }) {
  const precoFormatado = preco.toLocaleString("pt-BR", {
    style: "currency",
    currency: "BRL",
  });

  return (
    <article className="card-prato">
      <span className="categoria">{categoria}</span>
      <h2>{nome}</h2>
      <p className="preco">{precoFormatado}</p>
    </article>
  );
}

export default CardPrato;
```

```

1  function CardPrato({ nome, preco, categoria }) {
2      const precoFormatado = preco.toLocaleString("pt-BR", {
3          style: "currency",
4          currency: "BRL",
5      });
6
7      return (
8          <article className="card-prato">
9              <span className="categoria">{categoria}</span>
10             <h2>{nome}</h2>
11             <p className="preco">{precoFormatado}</p>
12         </article>
13     );
14 }
15
16 export default CardPrato;
    
```

CardPrato com precoFormatado.

Agora o preço aparece no padrão brasileiro, com vírgula: R\$ 42,90.



Resultado final: R\$ 42,90 / R\$ 49,90 / R\$ 15,00.

O que mudou

Antes: `toFixed(2)` mostrava R\$ 42.90 (ponto). Agora: `toLocaleString("pt-BR", {style:"currency", currency:"BRL"})` mostra R\$ 42,90 (vírgula) e coloca o R\$ sozinho. Preparar o valor antes do return é boa prática.

- Meu preço aparece como R\$ 42,90 (com vírgula).

Travou? Confira isto

- Página não abre? Veja se o terminal com `npm run dev` ainda está aberto.
- Erro "must have one parent"? Envolve tudo do return em um só `<main>` / `<div>`.
- Warning "key"? Faltou o `key={prato.id}` no `.map()`.
- Tela branca? Erro de digitação — olhe a mensagem no terminal.
- Não conseguiu? Chame o professor.

No fim você tem

- Projeto React rodando no localhost.
- Um Header e um CardPrato (com props).
- O cardápio completo via `.map()` com `key`, com preço em R\$.